

# Lista de Exercícios: Redes Neurais Artificiais

## Do Neurônio de McCulloch-Pitts ao Perceptron Multicamadas (MLP)

Material Grounded com base exclusiva nas Notas de Aula de Inteligência Computacional Aplicada  
Prof. Dr. Guilherme de Alencar Barreto (DETI/UFC)

### Instruções para o Estudante

Esta lista contém **20 exercícios** de dificuldade progressiva divididos em cinco categorias. Todas as questões foram elaboradas com base estrita no material didático e notas de aula fornecidos (páginas 1 a 45 e os materiais de Perceptron, Adaline e MLP). Responda a cada questão justificando seus passos de maneira rigorosa e clara, utilizando as fórmulas matemáticas apresentadas no material.

## Parte I — Questões Conceituais

### 1. Mapeamento Biológico vs. Artificial no Neurônio MP

O neurônio artificial de McCulloch-Pitts (MP) foi proposto em 1943 como uma aproximação matemática das funcionalidades de processamento de informação de um neurônio biológico real. Com base estrita nas fontes estudadas, explique:

- Como as partes fundamentais do neurônio biológico — sinapses, dendritos, corpo celular (soma) e axônio — são mapeadas nas variáveis e operações matemáticas do neurônio artificial MP?
- Qual a função e interpretação matemática dada ao modelar o axônio como uma chave do tipo ON-OFF (função degrau/sinal) e qual é o impacto desse tipo de saída sobre as categorias de problemas de reconhecimento de padrões que o neurônio MP pode solucionar? Justifique com base nas referências.

### 2. A Transição de Não-Linearidades Duras para Suaves (Degrau vs. Sigmoide)

No Perceptron Multicamadas (MLP), a função de ativação degrau tradicional do neurônio MP é substituída por funções de ativação suaves do tipo sigmoideal (como a sigmoide logística ou a tangente hiperbólica).

- Apresente as vantagens conceituais e matemáticas dessa substituição, explicando de que forma as saídas analógicas (reais) em vez de binárias/bipolares (ON-OFF) enriquecem a interpretação biológica e algorítmica da rede.
- Explique de forma detalhada uma desvantagem computacional associada ao uso de funções de ativação sigmoidais em sistemas embarcados e como isso se relaciona matematicamente com a sua expansão em série de Taylor.

### 3. Paralisia da Rede (Network Paralysis)

O fenômeno da "paralisia da rede" representa um grande obstáculo no treinamento do Perceptron Multicamadas (MLP).

- Com base nas equações da derivada da sigmoide logística ( $y'_i = y_i(1 - y_i)$ ), explique de forma rigorosa por que o ajuste de pesos se torna extremamente lento quando a ativação líquida ( $u_i$  ou  $a_k$ ) é muito alta ou muito baixa.

- b) Apresente e explique as três dicas práticas fornecidas no material de aula para mitigar a paralisa da rede no início do treinamento, demonstrando matematicamente como a adição de uma constante ao gradiente local altera a regra de modificação de pesos.
4. **Capacidade de Aproximação Universal: Redes de Uma vs. Duas Camadas Ocultas**  
Com base nos teoremas de existência clássicos descritos no material (ex. Hornik et al., Hecht-Nielsen), responda:
- a) Qual é a diferença na capacidade de aproximação universal entre uma rede MLP com uma única camada oculta e uma rede MLP com duas camadas ocultas? Classifique os tipos de funções que cada arquitetura é capaz de aproximar com grau de precisão arbitrário.
- b) Explique o papel fundamental que as funções de ativação não-lineares desempenham nessa capacidade de aproximação. O que aconteceria à estrutura matemática geral da rede se essas funções fossem removidas das camadas ocultas? Demonstre algebricamente usando matrizes de pesos.

## Parte II — Questões de Análise e Interpretação

### 5. Análise Geométrica e Fronteira de Decisão da Porta OR

Considere um neurônio de McCulloch-Pitts projetado para implementar a porta lógica OR bipassar (saídas  $\in \{-1, +1\}$ ), cujas entradas são  $x_1, x_2 \in \{0, 1\}$ . Os pesos foram definidos pelo projetista como  $w_1 = w_2 = 1$  e o limiar (bias) como  $\theta = 0,5$ .

- a) Realize a verificação algébrica do neurônio para cada uma das quatro combinações de entrada, apresentando os valores de ativação  $u$  e saída  $y = \text{sin}(u)$ , onde  $y = +1$  se  $u \geq 0$  e  $y = -1$  se  $u < 0$ .
- b) Determine a reta de separação das classes no espaço cartesiano  $(x_1, x_2)$ . Calcule os pontos exatos onde essa reta de fronteira de decisão corta os eixos coordenados e prove graficamente que a reta separa perfeitamente a Classe 1 ( $y = +1$ ) da Classe 2 ( $y = -1$ ).

### 6. Inseparabilidade Linear do XOR e Mapeamento de Espaço Oculto

A porta lógica XOR é conhecida por ser um problema não-linearmente separável. No entanto, uma rede neural multicamadas consegue resolvê-lo por meio de um mapeamento não-linear.

- a) Demonstre graficamente o problema do XOR nas coordenadas de entrada  $(x_1, x_2) \in \{(0, 0), (0, 1), (1, 0), (1, 1)\}$  e explique por que é geometricamente impossível separá-lo com uma única reta.
- b) O material de slides propõe a introdução de dois neurônios intermediários ( $z_1$  correspondente à porta AND e  $z_2$  correspondente à porta OR). Apresente a tabela do mapeamento das quatro entradas do XOR para o novo espaço de representação  $(z_1, z_2)$  e analise de forma justificada por que nesse novo espaço o terceiro neurônio de saída consegue realizar a classificação linear.

### 7. Treinamento Manual Época-a-Época do Perceptron Simples

Um Perceptron Simples é utilizado para aprender a porta lógica OR bipolar com entradas  $x_1, x_2 \in \{0, 1\}$  e saídas desejadas  $d \in \{0, 1\}$ . Os parâmetros iniciais de peso são nulos:  $w(0) = [\theta(0), w_1(0), w_2(0)]^T = [0, 0, 0]^T$  e o vetor de entrada expandido é  $x(t) = [-1, x_1(t), x_2(t)]^T$ . A taxa de aprendizagem é  $\eta = 0,5$ . Realize os cálculos manuais detalhados para as seguintes iterações de treinamento:

- a) Instante  $t = 1$ : Apresentação da entrada  $(x_1, x_2) = (0, 0)$  com saída desejada  $d = 0$ . Calcule  $u(1)$ ,  $y(1)$ , o erro  $e(1)$  e os pesos atualizados  $w(2)$ .

- b) Instante  $t = 2$ : Apresentação da entrada  $(x_1, x_2) = (0, 1)$  com saída desejada  $d = 1$ . Calcule  $u(2)$ ,  $y(2)$ , o erro  $e(2)$  e os pesos atualizados  $w(3)$ .
- c) Analise os resultados de  $w(2)$  e  $w(3)$ . Por que ocorreu modificação nos pesos em  $t = 2$  mas não em  $t = 1$ ? Justifique sob a ótica da regra de aprendizagem.

### 8. Análise Comparativa de Funções-Objetivo: Perceptron vs. Adaline

A regra de aprendizagem do Perceptron Simples e a regra de aprendizagem do Adaline (regra delta/LMS) são baseadas na minimização de critérios matemáticos distintos.

- a) Apresente as equações das funções-objetivo (ou funções de custo) de interesse  $J[w]$  para o Perceptron Simples e para o Adaline, detalhando as variáveis que as compõem.
- b) Analise a diferença fundamental em relação ao momento em que o erro  $e(t)$  é calculado em cada um desses dois algoritmos. Qual é a consequência prática desse cálculo na atualização de pesos para um padrão que foi classificado corretamente (saída final correta), mas cujo sinal de ativação contínua  $u(t)$  está distante da saída desejada no caso do Adaline?

## Parte III — Questões de Implementação Computacional (em R)

### 9. Completar Algoritmo do Perceptron Simples em R

O código abaixo implementa uma época de treinamento de uma rede Perceptron Simples com saídas bipolares  $y \in \{-1, +1\}$  e uma taxa de aprendizagem  $\alpha = 0.5$ . Substitua as lacunas denotadas por `___(lacuna)___` com as linhas correspondentes em código R de modo a calcular a saída estimada utilizando a função sinal, determinar o erro, e atualizar os pesos utilizando as equações do material.

```

1 perceptron_treinar_epoca <- function(X, d, w, alpha = 0.5) {
2   # X: matriz de entrada expandida (N x (p+1)), onde a primeira coluna e
      -1 (bias)
3   # d: vetor de sa das desejadas (N x 1)
4   # w: vetor de pesos inicial ((p+1) x 1)
5   N <- nrow(X)
6
7   for (t in 1:N) {
8     x_t <- X[t, ] # vetor de entrada para o exemplo atual
9
10    # 1. Calcular a ativa o (produto escalar)
11    u <- sum(w * x_t)
12
13    # 2. Calcular a sa da gerada via fun o sinal (bipolar)
14    # LACUNA 1: Sa da y deve ser +1 se u >= 0, ou -1 caso contr rio
15    y <- ___(LACUNA_1)___
16
17    # 3. Calcular o erro do neur nio
18    # LACUNA 2: Equa o do erro correspondente ao Perceptron
19    e <- ___(LACUNA_2)___
20
21    # 4. Ajustar os pesos se houver erro
22    if (e != 0) {
23      # LACUNA 3: Regra de aprendizagem do Perceptron Simples
24      w <- ___(LACUNA_3)___
25    }
26  }
27  return(w)
28 }
```

Explique detalhadamente qual é a representação vetorial da linha de ajuste de pesos da Lacuna 3 e a compatibilidade matemática dos vetores.

#### 10. Implementação Matricial LMS (Adaline/MADALINE)

De acordo com o material sobre o Adaline, a regra LMS sob notação matricial para múltiplos neurônios de saída (rede MADALINE) permite a atualização da matriz de pesos de uma única vez, aumentando a velocidade de execução do algoritmo. A regra matricial é dada por:

$$W(t+1) = W(t) + \alpha e(t)x^T(t)$$

Escreva uma linha de código em R que execute exatamente essa operação para uma iteração  $t$ . Considere que no ambiente as seguintes variáveis já estão definidas:

- **W**: Matriz de pesos de dimensão  $q \times (p+1)$ .
- **alpha**: Escalar que representa a taxa de aprendizagem  $\alpha$ .
- **e**: Vetor coluna de erros dos neurônios de saída de dimensão  $q \times 1$ .
- **x**: Vetor coluna correspondente à entrada expandida do modelo de dimensão  $(p+1) \times 1$ .

Justifique por que o operador de multiplicação matricial do R (`%*%`) e a função de transposição de matrizes (`t()`) são necessários, discutindo a dimensionalidade dos termos envolvidos.

#### 11. Função de Normalização de Dados

O material indica que para projetar uma rede Perceptron ou Adaline com variáveis que apresentem ordens de grandeza díspares, recomenda-se normalizar os dados.

- a) Escreva uma função em R chamada `normalizar_unipolar(x)` que receba um vetor numérico **x** e realize a normalização de modo a manter todos os valores dentro da faixa  $[0, +1]$  utilizando a fórmula:

$$x_j^{norm} = \frac{x_j - x^{min}}{x^{max} - x^{min}}$$

- b) Modifique essa função para criar `normalizar_bipolar(x)` para mapear os valores na faixa  $[-1, +1]$  usando a fórmula correspondente:

$$x_j^{norm} = 2 \cdot \left( \frac{x_j - x^{min}}{x^{max} - x^{min}} \right) - 1$$

#### 12. Codificação de Funções Sigmoidais e suas Derivadas

As funções de ativação suave de trecho central quase linear são fundamentais para o algoritmo backpropagation. Escreva duas funções em R:

- a) `sigmoide_logistica(u)`: que calcula e retorna a saída da função logística  $y = \frac{1}{1+e^{-u}}$ .
- b) `derivada_logistica(y)`: que recebe a saída de um neurônio logístico e calcula sua derivada parcial em relação à ativação através da fórmula simplificada  $y' = y(1-y)$ .

Utilize essas funções em R para calcular a saída e a derivada para as ativações  $u = -3$ ,  $u = 0$  e  $u = 3$ . Comente a relação entre o valor da derivada nesses pontos e a velocidade de ajuste de pesos discutida sob o ponto de vista teórico.

#### 13. Correção de Bug em Algoritmo de Atualização da Camada de Saída (MLP)

Um aluno tentou implementar a regra de aprendizagem dos neurônios de saída de uma rede MLP com uma camada oculta. Sua tentativa em R foi:

```

1 # Tentativa incorreta de implementar a regra:
2 #  $m_{k\_novo} = m_{k\_velho} + \eta * e_k * z_t$ 
3 atualizar_pesos_saida_incorreto <- function(M, eta, e, z) {
4   # M: matriz de pesos da camada de saída de dimensão  $c \times (q_1 + 1)$ 
5   # e: vetor coluna de erros ( $c \times 1$ )
6   # z: vetor de entradas da camada de saída (saídas da camada oculta, ( $q_1 + 1$ )  $\times$  1)
7
8   M_novo <- M + eta * e * z
9   return(M_novo)
10 }
```

Identifique os dois erros conceituais e de sintaxe de multiplicação de vetores/matrizes presentes no código acima que violam a teoria e a álgebra linear da regra de atualização dos pesos de saída de uma MLP de ativação logística:

$$m_k(t+1) = m_k(t) + \eta \delta_k(t) z(t), \quad \text{onde } \delta_k(t) = e_k(t) y'_k(t)$$

Escreva a versão corrigida da função `atualizar_pesos_saida_correto(M, eta, e, y_saida, z)` em R, garantindo a multiplicação matricial correta para atualização de toda a matriz de uma vez (como feito no Adaline e no Perceptron).

## Parte IV — Questões de Conexão

### 14. Conexão entre Produto Escalar e Ativação do Neurônio MP

A ativação do neurônio de McCulloch-Pitts na forma vetorial é escrita como  $u = w^T x$ , a qual representa matematicamente o produto escalar entre o vetor de entrada expandido  $x$  e o vetor de pesos  $w$ .

- Mostre de que forma os conceitos geométricos do produto escalar se aplicam na determinação da saída do neurônio, discutindo especificamente o ângulo ( $\alpha$ ) formado entre os dois vetores.
- Discuta detalhadamente as duas condições apresentadas no material: Caso 1 ( $0 \leq \alpha < 90^\circ$ , implicando em  $\cos(\alpha) > 0 \Rightarrow u > 0$ ) e Caso 2 ( $90^\circ < \alpha \leq 180^\circ$ , implicando em  $\cos(\alpha) < 0 \Rightarrow u < 0$ ). Conecte o sinal da ativação  $u$  com a saída final de classificação  $y(t) \in \{-1, +1\}$  [19].

### 15. Conexão entre Parâmetros, Arquitetura e Complexidade (Viés-Variância)

A representação arquitetural de uma MLP com uma camada oculta é dada por  $MLP(p, q_1, c)$ , enquanto com duas camadas ocultas é dada por  $MLP(p, q_1, q_2, c)$ .

- Demonstre como deduzir matematicamente as duas fórmulas para o cálculo do número total de parâmetros ajustáveis  $Z$  (pesos e limiares) fornecidas no material:

$$Z_1 = (p+1)q_1 + (q_1+1)c$$

$$Z_2 = (p+1)q_1 + (q_1+1)q_2 + (q_2+1)c$$

- Considere o problema prático de auxílio ao diagnóstico de dermatologia apresentado no material, o qual possui  $p = 34$  atributos clínicos e histopatológicos de entrada e  $C = 6$  classes de saída (patologias). Se decidirmos utilizar uma arquitetura com uma única camada oculta contendo  $q_1 = 15$  neurônios intermediários, determine o número exato de parâmetros  $Z$ . Com base no "Dilema Viés-Variância" apontado no material, o que ocorreria com o desempenho se  $q_1$  fosse excessivamente pequeno ou excessivamente grande?

### 16. Conexão entre a Teoria Geométrica de Ajuste e a Regra do Perceptron

A regra de aprendizagem do Perceptron Simples, baseada em argumentos geométricos, dita que:

- Se ocorre uma classificação incorreta na qual a saída desejada é maior que a obtida ( $e(t) = +1$ ), o vetor de pesos atual deve ser somado a um incremento proporcional à entrada:  $w(t+1) = w(t) + \eta x(t)$  (Caso 1 do material).
- Se ocorre classificação incorreta na qual a saída desejada é menor que a obtida ( $e(t) = -1$ ), o vetor de pesos atual deve ser subtraído da entrada:  $w(t+1) = w(t) - \eta x(t)$  (Caso 2 do material).

Discuta como essas ações geométricas alteram a orientação espacial do hiperplano (vetor de pesos) em relação ao padrão mal-classificado  $x(t)$  (aproximando ou afastando o vetor de pesos do vetor de entrada) de forma a garantir que a próxima apresentação desse padrão reduza a chance de um novo erro. Conecte essa descrição com a equação sintética compactada:

$$w(t+1) = w(t) + \eta e(t)x(t)$$

### 17. Conexão entre Divisão de Dados, Critérios de Parada e Generalização

O treinamento de algoritmos adaptativos (como o Perceptron e o Adaline) é guiado de forma iterativa por meio de apresentações repetidas de dados divididos em conjuntos de "treinamento" e "teste".

- Explique de forma articulada a relação existente entre o critério de parada do algoritmo (EQM mínimo alcançado,  $J(t) < J_{min}$ , ou limite de épocas  $K_{max}$ ) e a capacidade de generalização do modelo adaptativo.
- Por que o erro de generalização obtido no conjunto de teste é conceitualmente e tipicamente maior que o erro de treinamento do conjunto ajustado? Discuta a relevância de embaralhar a ordem de apresentação dos padrões a cada época de treinamento, conforme as boas práticas do material de aula.

## Parte V — Questões de Explicação Aprofundada

### 18. Derivação Teórica Passo a Passo do Algoritmo Backpropagation (Camada de Saída)

A regra de aprendizagem do algoritmo backpropagation pode ser obtida por meio da minimização iterativa do erro quadrático instantâneo da camada de saída via gradiente descendente. O erro quadrático instantâneo para todos os  $c$  neurônios de saída é definido por:

$$J(t) = \frac{1}{2} \sum_{k=1}^c e_k^2(t) = \frac{1}{2} \sum_{k=1}^c (d_k(t) - y_k(t))^2$$

Com base estrita nas passagens teóricas fornecidas pelas notas de aula:

- Escreva a equação de atualização recursiva do vetor de pesos  $m_k(t)$  (neurônio  $k$  de saída) baseada no gradiente da função de custo  $J(t)$  na direção de  $m_k(t)$ , explicando o significado do termo do gradiente.
- Utilizando a regra da cadeia, demonstre detalhadamente a fatoração da derivada parcial em termos elementares:

$$\frac{\partial J(t)}{\partial m_k(t)} = \frac{\partial J(t)}{\partial e_k(t)} \cdot \frac{\partial e_k(t)}{\partial y_k(t)} \cdot \frac{\partial y_k(t)}{\partial a_k(t)} \cdot \frac{\partial a_k(t)}{\partial m_k(t)}$$

- c) Calcule explicitamente cada uma das quatro derivadas parciais sabendo que  $e_k = d_k - y_k$ ,  $y_k = \varphi(a_k)$  (sigmoide genérica de derivada  $\varphi'$ ) e  $a_k = m_k^T z(t)$ . Conclua a obtenção teórica da regra demonstrando a equação final:

$$m_k(t+1) = m_k(t) + \eta \delta_k(t) z(t)$$

### 19. O Mecanismo de Erro "Inventado" do Backpropagation para a Camada Oculta

Diferentemente do neurônio de saída, os neurônios ocultos de uma rede MLP não possuem acesso a um sinal de saída desejado  $d_i$  fornecido por um "supervisor" externo. Explique detalhadamente como esse problema é solucionado matematicamente pelo algoritmo Backpropagation, abordando:

- a) Como o erro do  $i$ -ésimo neurônio oculto ( $e_i(t)$ ) é definido ou "inventado" a partir da retroprojeção (retropropagação) dos gradientes locais  $\delta_k(t)$  da camada seguinte. Apresente e detalhe o significado físico de cada componente da equação:

$$e_i(t) = \sum_{k=1}^c m_{ki}(t) \delta_k(t)$$

- b) Como o gradiente local do neurônio oculto ( $\delta_i(t)$ ) é formulado para funções de ativação do tipo sigmoide logística e tangente hiperbólica.
- c) Escreva o passo a passo da regra final de atualização dos pesos ocultos  $w_i(t+1) = w_i(t) + \eta \delta_i(t) x(t)$  demonstrando de que forma as modificações dependem estritamente do vetor de entrada expandido original  $x(t)$ .

### 20. Análise Matemática de Dimensionamento com a Heurística de Baum-Haussler

Uma das técnicas heurísticas mais importantes para determinar de forma preliminar se a dimensão da camada oculta é viável em termos de generalização é a Regra de Baum-Haussler, baseada na inequação:

$$N > \frac{Z}{\epsilon}$$

Onde  $N$  representa o tamanho do conjunto de dados de treinamento,  $Z$  é o número total de parâmetros livres e ajustáveis da rede  $MLP(p, q_1, c)$  com uma camada oculta, e  $\epsilon$  é o erro máximo tolerado durante a fase de teste. Considere os seguintes dados de um problema específico:

- Número de variáveis de entrada:  $p = 10$ .
  - Número de neurônios na camada de saída:  $c = 2$ .
  - Erro máximo tolerado no teste: 10% (ou seja,  $\epsilon = 0,1$ ).
- a) Calcule a expressão do número total de parâmetros livres  $Z$  em função de  $q_1$  (número de neurônios ocultos).
- b) Se o conjunto de dados de treinamento disponível contiver exatamente  $N = 1000$  exemplos de casos reais, determine qual é o número máximo de neurônios ocultos  $q_1$  permitidos para satisfazer de forma rigorosa a inequação de Baum-Haussler.
- c) Discuta por que violar esse limite superior de neurônios ocultos aumenta a chance de insucesso de generalização da rede no teste com dados não vistos.

## Gabarito Detalhado

### Respostas Esperadas e Raciocínios Matemáticos Grounded

---

#### Parte I — Questões Conceituais

##### 1. Mapeamento Biológico vs. Artificial no Neurônio MP

- a) De acordo com os slides 5 e 6 do material, as partes estruturais são mapeadas da seguinte forma:
- **Dendritos:** Modelados pelas variáveis de entrada  $x_1, x_2, \dots, x_p$ . São responsáveis por trazer os potenciais de ação até o neurônio.
  - **Sinapses:** Modeladas pelos pesos sinápticos  $w_1, w_2, \dots, w_p$ . Representam a força de conexão e transmitem a informação de forma ponderada.
  - **Corpo Celular:** Modelado pelo somador e limiar de ativação. A ativação líquida é calculada por  $u = \sum_{j=1}^p w_j x_j - \theta$ . O limite de disparo do neurônio é representado pela subtração do limiar  $\theta$ .
  - **Axônio:** Modelado pela função de ativação que define a saída  $y$ . Funciona como uma chave ON-OFF determinando se o neurônio dispara potencial de ação.
- b) O axônio é modelado matematicamente como uma função sinal ou degrau duro ( $y = \text{sin}(u) = +1$  se  $u > 0$  e  $-1$  ou  $0$  se  $u \leq 0$ ). No slide 15, detalha-se que isso limita a aplicação do neurônio MP a problemas de reconhecimento de padrões binários ou bipolares (duas categorias). Ele atua puramente como um classificador linear ótimo capaz de lidar apenas com classes linearmente separáveis por reta, plano ou hiperplano.

##### 2. A Transição de Não-Linearidades Duras para Suaves (Degrâu vs. Sigmoides)

- a) Segundo o slide 51, a substituição da ativação rígida (degrâu) por ativações do tipo sigmoide (logística ou tangente hiperbólica) remove as descontinuidades e introduz uma não-linearidade suave (soft). Conceitualmente, a saída  $y_i$  deixa de ser discreta (0/1 ou -1/+1) e passa a ser contínua analógica, assumindo qualquer valor real no intervalo (0, 1) ou (-1, 1). Isso permite interpretar a saída do neurônio como uma "taxa média de disparo" (mean firing rate), aproximando o modelo artificial do comportamento analógico dos disparos em frequências dos neurônios biológicos reais.
- b) A grande desvantagem apontada no slide 53 é o elevado custo computacional para implementação física em sistemas embarcados de tempo real devido à presença do termo exponencial ( $\exp(-u_i)$ ). Em hardware de baixa capacidade, o cálculo do termo  $\exp(x)$  exige aproximações em série de Taylor, demandando múltiplos ciclos de clock para realizar multiplicações e somas sucessivas:  $\exp(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \dots$

##### 3. Paralisia da Rede (Network Paralysis)

- a) Conforme detalhado nos slides 68 e 69, se o valor de ativação líquida  $a_k(t)$  ou  $u_i(t)$  for muito alto (aproximando-se de  $+\infty$ ) ou muito baixo (aproximando-se de  $-\infty$ ), a saída  $y_i(t)$  da rede estará na região de saturação da função logística (aproximando-se de 1 ou 0, respectivamente). Quando isso ocorre, a derivada logística  $y'_i(t) = y_i(t)(1 - y_i(t))$  tende a zero. Visto que as equações de ajuste de pesos em ambas as camadas dependem diretamente de  $y'_i(t)$  para calcular o gradiente local  $\delta$  (onde  $\Delta w \propto \delta \propto y'$ ), os incrementos de pesos  $\Delta m_k(t)$  ou  $\Delta w_i(t)$  aproximam-se de zero, paralisando a evolução e desacelerando criticamente o processo de treinamento.
- b) Para mitigar o efeito da paralisia da rede, o slide 70 indica três heurísticas:

- (a) Iniciar pesos e limiares com valores pequenos (ex. na faixa  $[-0.1, +0.1]$  ou  $[-0.5, +0.5]$ ) para manter as ativações iniciais na faixa linear central da sigmoide (onde a derivada é máxima, próximo a 0,25).
- (b) Normalizar as variáveis de entrada do modelo para que as escalas não fiquem excessivamente díspares.
- (c) Modificar a regra de ajuste adicionando uma constante igual a 0,05 ao gradiente local dos neurônios, forçando um patamar mínimo para a variação dos pesos:

$$\Delta m_k(t) = \eta e_k(t)(\delta_k(t) + 0,05)z(t)$$

$$\Delta w_i(t) = \eta e_i(t)(\delta_i(t) + 0,05)x(t)$$

#### 4. Capacidade de Aproximação Universal: Redes de Uma vs. Duas Camadas Ocultas

- a) De acordo com o slide 65, os teoremas clássicos demonstram que:
  - **MLP de uma camada oculta:**  $MLP(p, q_1, c)$  é capaz de aproximar qualquer função **contínua** com grau de precisão arbitrário, sob a condição de possuir um número suficientemente grande de neurônios ocultos com ativação sigmoide.
  - **MLP de duas camadas ocultas:**  $MLP(p, q_1, q_2, c)$  é capaz de aproximar qualquer função **descontínua** com grau de precisão arbitrário, dado um número suficientemente grande de neurônios com ativação sigmoide em cada camada.
- b) As não-linearidades suaves introduzidas pelas funções de ativação entre as camadas ocultas são responsáveis pelo poder de aproximação universal. Sem a função de ativação não-linear entre camadas sucessivas (slide 48), a propagação de sinais de uma MLP de uma camada oculta reduzir-se-ia a:

$$y = \varphi(a) = \varphi(Mz) = \varphi(M\varphi(Wx)) \Rightarrow \text{sem a não-linearidade: } y = M(Wx) = (MW)x = Kx$$

Como a multiplicação de duas matrizes lineares de pesos resulta em uma única matriz equivalente  $K = M \cdot W$ , a rede inteira comportar-se-ia como uma rede linear Perceptron Simples, perdendo toda a capacidade de representar fronteiras de decisão complexas e não-lineares.

## Parte II — Questões de Análise e Interpretação

### 1. Análise Geométrica e Fronteira de Decisão da Porta OR

- a) Para realizar a verificação do neurônio, calculamos a ativação  $u = (w_1x_1) + (w_2x_2) - \theta = 1 \cdot x_1 + 1 \cdot x_2 - 0,5$  e aplicamos a função sinal (degrau duro) descrita nos slides 11 e 12:
  - i)  $(x_1, x_2) = (0, 0) \Rightarrow u = 1(0) + 1(0) - 0,5 = -0,5 < 0 \Rightarrow y = -1$  (Classe 2, correta).
  - ii)  $(x_1, x_2) = (1, 0) \Rightarrow u = 1(1) + 1(0) - 0,5 = +0,5 \geq 0 \Rightarrow y = +1$  (Classe 1, correta).
  - iii)  $(x_1, x_2) = (0, 1) \Rightarrow u = 1(0) + 1(1) - 0,5 = +0,5 \geq 0 \Rightarrow y = +1$  (Classe 1, correta).
  - iv)  $(x_1, x_2) = (1, 1) \Rightarrow u = 1(1) + 1(1) - 0,5 = +1,5 \geq 0 \Rightarrow y = +1$  (Classe 1, correta).
- b) A fronteira de decisão de um neurônio artificial é obtida fazendo  $u = 0$ , resultando na equação da reta (slide 8):

$$1 \cdot x_1 + 1 \cdot x_2 - 0,5 = 0 \Rightarrow x_2 = -x_1 + 0,5$$

Os pontos de corte nos eixos coordenados são dados de acordo com o slide 11 por:

- Corte no eixo  $x_2$  (fazendo  $x_1 = 0$ ):  $(0, \theta/w_2) = (0, 0,5/1) = (0, 0,5)$ .
- Corte no eixo  $x_1$  (fazendo  $x_2 = 0$ ):  $(\theta/w_1, 0) = (0,5/1, 0) = (0,5, 0)$ .

Esboçando os pontos  $(0, 0)$  como classe negativa (marcada como círculo branco) e os pontos  $(1, 0), (0, 1), (1, 1)$  como classe positiva (marcada como círculos pretos), a reta diagonal passando por  $(0, 5, 0)$  e  $(0, 0, 5)$  isola perfeitamente o ponto da origem do resto dos padrões, provando que o problema é linearmente separável.

### 5. Inseparabilidade Linear do XOR e Mapeamento de Espaço Oculto

- a) No plano cartesiano  $(x_1, x_2)$ , os pontos do XOR bipolar correspondentes à classe positiva ( $y = 1$ ) localizam-se em  $(1, 0)$  e  $(0, 1)$ , enquanto os pontos da classe negativa ( $y = 0$ ) localizam-se em  $(0, 0)$  e  $(1, 1)$ . Se tentarmos traçar uma reta que isole os pontos positivos, inevitavelmente um ponto negativo ficará do mesmo lado, pois os padrões positivos estão dispostos diagonalmente em relação aos negativos.
- b) Introduzindo dois neurônios intermediários cujas ativações correspondem a  $z_1 = AND(x_1, x_2)$  e  $z_2 = OR(x_1, x_2)$ , temos a seguinte tabela de mapeamento para o espaço oculto de representação  $(z_1, z_2)$  de acordo com as especificações dos slides 41 e 42:

| $x_1$ | $x_2$ | $z_1 = AND(x_1, x_2)$ | $z_2 = OR(x_1, x_2)$ | Saída Desejada $y$ |
|-------|-------|-----------------------|----------------------|--------------------|
| 0     | 0     | 0                     | 0                    | 0                  |
| 0     | 1     | 0                     | 1                    | 1                  |
| 1     | 0     | 0                     | 1                    | 1                  |
| 1     | 1     | 1                     | 1                    | 0                  |

No novo espaço bidimensional definido pelas variáveis ocultas  $(z_1, z_2)$ , os pontos mapeados são:

- Entrada  $(0, 0) \rightarrow (z_1, z_2) = (0, 0)$  (Classe  $y = 0$ ).
- Entradas  $(0, 1)$  e  $(1, 0) \rightarrow (z_1, z_2) = (0, 1)$  (Classe  $y = 1$ ).
- Entrada  $(1, 1) \rightarrow (z_1, z_2) = (1, 1)$  (Classe  $y = 0$ ).
- O quarto quadrante virtual  $(1, 0)$  não possui dados mapeados sobre si.

No espaço cartesiano  $(z_1, z_2)$ , os dois pontos de classe positiva colapsam em uma única coordenada  $(0, 1)$ . Os pontos da classe negativa ocupam os vértices opostos  $(0, 0)$  e  $(1, 1)$ . Conforme demonstrado no slide 43, agora as duas classes são linearmente separáveis por meio de uma reta simples que separe o ponto  $(0, 1)$  das coordenadas restantes.

### 6. Treinamento Manual Época-a-Época do Perceptron Simples

- a) **Instante**  $t = 1$ : pesos iniciais  $w(1) = [0, 0, 0]^T$ . Entrada  $x(1) = [-1, 0, 0]^T$  com saída desejada  $d(1) = 0$ .
- Ativação:  $u(1) = w(1)^T x(1) = (0)(-1) + (0)(0) + (0)(0) = 0$ .
  - Saída gerada:  $y(1) = 0$  (visto que  $u \geq 0$ , as saídas da porta lógica dadas pelas especificações do slide 25 e 26 indicam resposta 0).
  - Erro do neurônio:  $e(1) = d(1) - y(1) = 0 - 0 = 0$ .
  - Atualização de pesos: como o erro é nulo,  $w(2) = w(1) + \eta e(1)x(1) = [0, 0, 0]^T$ .
- b) **Instante**  $t = 2$ : pesos atuais  $w(2) = [0, 0, 0]^T$ . Entrada  $x(2) = [-1, 0, 1]^T$  com saída desejada  $d(2) = 1$ .
- Ativação:  $u(2) = w(2)^T x(2) = (0)(-1) + (0)(0) + (0)(1) = 0$ .
  - Saída gerada:  $y(2) = 0$  (visto que  $u \geq 0$ , como o valor desejado para esta entrada é 1, o neurônio disparou abaixo do limite esperado).
  - Erro do neurônio:  $e(2) = d(2) - y(2) = 1 - 0 = 1$ .
  - Atualização de pesos:

$$w(3) = w(2) + \eta e(2)x(2) = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + 0,5 \cdot (1) \cdot \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -0,5 \\ 0 \\ 0,5 \end{bmatrix}^T$$

- c) No instante  $t = 1$ , a classificação do Perceptron gerou saída igual à desejada ( $y(1) = d(1) = 0$ ), resultando em erro nulo e nenhuma variação nos pesos. Isso está de acordo com o slide 23, que descreve que, se houver acerto, nenhuma alteração de memória é necessária. No instante  $t = 2$ , houve classificação incorreta ( $y(2) = 0$  mas  $d(2) = 1$ ), gerando erro igual a +1. Com base na equação de aprendizagem, o vetor de pesos modificou-se na direção do vetor de entrada para aumentar o valor da ativação líquida nas próximas apresentações desse mesmo padrão.

## 7. Análise Comparativa de Funções-Objetivo: Perceptron vs. Adaline

- a) Com base nos materiais estudados:

- **Perceptron Simples (Eq. 11 das notas de Perceptron):**

$$J[w] = \sum_{z_k \in Z} (-w^T z_k)$$

Onde  $Z$  é o subconjunto que agrupa exclusivamente os vetores de entrada  $z_k$  que foram mal-classificados pelo Perceptron atual.

- **Adaline (Eq. 11 do Adaline):**

$$J[W] = \frac{1}{2N} \sum_{t=1}^N e^2(t) = \frac{1}{2N} \sum_{t=1}^N [d(t) - u(t)]^2$$

Onde  $N$  é o número total de exemplos de treinamento e  $u(t)$  é a ativação analógica (produto escalar contínuo) do neurônio Adaline no instante  $t$ .

- b) A diferença crucial é que o erro do Perceptron é calculado **após a quantização** da saída pela função sinal ou degrau duro ( $e(t) = d(t) - y(t)$ ), enquanto o erro do Adaline é medido **antes da quantização**, utilizando diretamente o sinal analógico de ativação ( $e(t) = d(t) - u(t)$ ). Se um padrão foi classificado corretamente ( $y(t) = d(t)$ ), o erro do Perceptron será nulo e não alterará seus pesos. No caso do Adaline, se a ativação  $u(t)$  estiver distante da saída desejada contínua (ex.  $u(t) = 3$  para  $d(t) = 1$ ), o erro residual será  $e(t) = 1 - 3 = -2 \neq 0$ . Assim, mesmo com classificação correta pelo quantizador, o Adaline continuará refinando e modificando os pesos em direção à minimização do erro quadrático contínuo.

## Parte III — Questões de Implementação Computacional (em R)

### 1. Completar Algoritmo do Perceptron Simples em R

- **Preenchimento das Lacunas:**

- LACUNA\_1: `ifelse(u >= 0, 1, -1)`
- LACUNA\_2: `d[t] - y`
- LACUNA\_3: `w + alpha * e * x_t`

- **Raciocínio Matemático e Dimensional:** A linha de pesos atualizados reflete a equação  $w(t+1) = w(t) + \alpha e(t)x(t)$ . O termo  $\alpha e(t)x(t)$  é a multiplicação de um escalar ( $\alpha e(t)$ ) pelo vetor de entrada expandido  $x(t)$ , que tem dimensões  $((p+1) \times 1)$  idênticas às de  $w(t)$ . Assim, a soma elemento a elemento é válida e resulta em um novo vetor de pesos atualizado.

### 9. Implementação Matricial LMS (Adaline/MADALINE)

- **Linha de Código em R:**

```
1 W <- W + alpha * e %*% t(x)
```

- **Justificativa Dimensional:** A matriz de pesos original  $W$  tem dimensão  $q \times (p + 1)$ . O vetor de erros  $e(t)$  tem dimensão  $q \times 1$  e o vetor de entrada expandido  $x(t)$  tem dimensão  $(p + 1) \times 1$ . A transposição  $t(x)$  produz um vetor linha de dimensão  $1 \times (p + 1)$ . O produto externo obtido pela multiplicação matricial  $\%*\%$  entre o vetor coluna  $e$  ( $q \times 1$ ) e o vetor linha  $t(x)$  ( $1 \times (p + 1)$ ) resulta em uma matriz de dimensões idênticas à de pesos:  $q \times (p + 1)$ . Isso permite somar e atualizar os pesos de todas as sinapses da rede de forma simultânea e vetorizada.

## 10. Função de Normalização de Dados

- a) Função R para normalização unipolar  $[0, +1]$ :

```

1 normalizar_unipolar <- function(x) {
2   min_val <- min(x)
3   max_val <- max(x)
4   # Evita divisao por zero caso o vetor seja constante
5   if (max_val == min_val) return(rep(0.5, length(x)))
6   x_norm <- (x - min_val) / (max_val - min_val)
7   return(x_norm)
8 }

```

- b) Função R para normalização bipolar  $[-1, +1]$ :

```

1 normalizar_bipolar <- function(x) {
2   min_val <- min(x)
3   max_val <- max(x)
4   if (max_val == min_val) return(rep(0, length(x)))
5   x_norm <- 2 * ((x - min_val) / (max_val - min_val)) - 1
6   return(x_norm)
7 }

```

## 11. Codificação de Funções Sigmoidais e suas Derivadas

- **Implementação em R:**

```

1 sigmoide_logistica <- function(u) {
2   return(1 / (1 + exp(-u)))
3 }
4
5 derivada_logistica <- function(y) {
6   return(y * (1 - y))
7 }

```

- **Cálculos:**

- Para  $u = -3 \Rightarrow y = \frac{1}{1+e^3} \approx 0,0474$ ;  $y' = 0,0474 \cdot (1 - 0,0474) \approx 0,0452$ .
- Para  $u = 0 \Rightarrow y = \frac{1}{1+e^0} = 0,5$ ;  $y' = 0,5 \cdot (1 - 0,5) = 0,25$  (valor máximo).
- Para  $u = 3 \Rightarrow y = \frac{1}{1+e^{-3}} \approx 0,9526$ ;  $y' = 0,9526 \cdot (1 - 0,9526) \approx 0,0452$ .

- **Comentário Teórico:** Nos pontos de saturação  $u = -3$  e  $u = 3$ , a derivada parcial assume um valor baixo ( $\approx 0,045$ ), reduzindo o incremento da variação de peso. Por outro lado, para  $u = 0$  (trecho central mais linear), a derivada assume seu valor máximo (0,25). Isso corrobora o exposto nos slides 68 e 69, provando que no início do aprendizado manter os sinais na zona linear acelera a velocidade e a convergência do ajuste dos pesos sinápticos.

## 12. Correção de Bug em Algoritmo de Atualização da Camada de Saída (MLP)

- **Identificação dos Erros:**

- (a) Erro 1 (Teórico): O código utiliza o termo de erro puro  $e_k = d_k - y_k$  diretamente para ajustar os pesos. No entanto, para redes treinadas com funções não-lineares suaves por gradiente descendente, a regra exige o gradiente local  $\delta_k = e_k \cdot y'_k$ , onde  $y'_k$  é a derivada da sigmoide logística. Desconsiderar a derivada ignora a influência da inclinação da função de ativação sobre a minimização iterativa do erro.
- (b) Erro 2 (Álgebra Linear): O produto simples  $e * z$  multiplicará os elementos de dois vetores com dimensões incompatíveis se não for transposto corretamente. O correto é usar o operador de produto matricial `%*%` aplicando a transposição adequada de forma a obter uma matriz correspondente para atualizar toda a matriz de pesos  $M$ .

• **Código Corrigido em R:**

```
1 atualizar_pesos_saida_correto <- function(M, eta, e, y_saida, z) {  
2   # M: matriz de dimensao c x (q1 + 1)  
3   # e: vetor coluna de erros (c x 1)  
4   # y_saida: vetor de saidas geradas pelos neuronios (c x 1)  
5   # z: vetor coluna de entrada da camada de saida ((q1 + 1) x 1)  
6  
7   # 1. Calcular a derivada da sigmoide logistica  
8   y_derivada <- y_saida * (1 - y_saida)  
9  
10  # 2. Calcular o gradiente local delta_k  
11  delta <- e * y_derivada # multiplicacao elemento a elemento (c x  
12    1)  
13  
14  # 3. Atualizar a matriz de pesos por meio de produto externo  
15  M_novo <- M + eta * (delta %*% t(z))  
16  return(M_novo)  
}
```

## Parte IV — Questões de Conexão

### 14. Conexão entre Produto Escalar e Ativação do Neurônio MP

- a) O produto escalar entre dois vetores  $w$  e  $x$  pode ser redefinido por meio da relação trigonométrica  $u = \|w\| \cdot \|x\| \cdot \cos(\alpha)$ , onde  $\|v\|$  representa o comprimento euclidiano do vetor e  $\alpha$  é o ângulo formado entre ambos. Como a magnitude dos vetores é sempre estritamente positiva, o sinal algébrico do produto escalar (e consequentemente o sinal da ativação  $u$ ) depende exclusivamente do valor de  $\cos(\alpha)$ .
- b) Conectando com as propriedades trigonométricas apresentadas no slide 19:
- **Caso 1** ( $0 \leq \alpha < 90^\circ$ ): Nesse intervalo, o cosseno é positivo ( $\cos(\alpha) > 0$ ), indicando que os vetores pesos e entrada apontam para direções similares (ângulo agudo). Como resultado, a ativação líquida  $u$  é positiva ( $u > 0$ ) e a saída final gerada pela função sinal do neurônio MP dispara para  $y = +1$ .
  - **Caso 2** ( $90^\circ < \alpha \leq 180^\circ$ ): Nesse intervalo, o cosseno é negativo ( $\cos(\alpha) < 0$ ), indicando que os vetores apontam para direções divergentes (ângulo obtuso). Consequentemente, a ativação líquida é negativa ( $u < 0$ ) e a saída final do classificador decai para  $y = -1$ .

### 15. Conexão entre Parâmetros, Arquitetura e Complexidade (Viés-Variância)

- a) Dedução das fórmulas de parâmetros ajustáveis  $Z$  (pesos e limiares):
- **Caso 1: MLP com 1 Camada Oculta ( $MLP(p, q_1, c)$ ):** Cada um dos  $q_1$  neurônios da camada oculta possui  $p$  pesos conectados às entradas, mais 1 parâmetro associado ao limiar de ativação (bias), totalizando  $(p + 1)$  parâmetros por neurônio oculto. Isso

gera um subtotal de  $(p+1)q_1$  parâmetros ajustáveis. De forma análoga, cada um dos  $c$  neurônios da camada de saída conecta-se às  $q_1$  saídas intermediárias mais 1 limiar, gerando  $(q_1+1)c$  parâmetros. Somando as duas parcelas, temos:  $Z_1 = (p+1)q_1 + (q_1+1)c$ .

- **Caso 2: MLP com 2 Camadas Ocultas ( $MLP(p, q_1, q_2, c)$ ):** Seguindo o mesmo raciocínio de contagem por camada, temos:  $(p+1)q_1$  parâmetros de entrada para a primeira oculta;  $(q_1+1)q_2$  parâmetros de conexões intermediárias entre a primeira e a segunda oculta; e  $(q_2+1)c$  parâmetros associados às conexões da segunda oculta para a camada de saída. O somatório de parâmetros livres resulta exatamente em:  $Z_2 = (p+1)q_1 + (q_1+1)q_2 + (q_2+1)c$ .

b) Cálculo para o problema dermatológico com  $p = 34$ ,  $q_1 = 15$  e  $c = 6$ :

$$Z_1 = (34+1)15 + (15+1)6 = 35 \cdot 15 + 16 \cdot 6 = 525 + 96 = 621 \text{ parâmetros livres}$$

De acordo com os dilemas fundamentais de estatística e redes adaptativas, se o número de parâmetros ajustáveis  $q_1$  for indefinido ou inadequado, o modelo falhará:

- Se  $q_1$  for excessivamente pequeno, a rede terá alto "viés" (erro de subajuste/underfitting), pois faltará flexibilidade e parâmetros livres para mapear a complexidade inerente da fronteira de decisão (fronteira rígida demais).
- Se  $q_1$  for excessivamente grande, a rede terá alta "variância" (erro de superajuste/overfitting), decorrente de um número de parâmetros desproporcional à quantidade de dados disponíveis. A rede simplesmente decorará o ruído do conjunto de dados de treinamento, apresentando desempenho excelente no ajuste, mas falhando miseravelmente no teste com dados não vistos.

## 16. Conexão entre a Teoria Geométrica de Ajuste e a Regra do Perceptron

- Quando ocorre  $e(t) = +1$ , significa que o neurônio gerou uma ativação fraca ( $y(t) = 0$  ou  $-1$  mas  $d(t) = +1$ ). Geometricamente, o vetor de pesos  $w(t)$  e o vetor de entrada  $x(t)$  estão afastados de forma incoerente. Para aumentar a similaridade e, conseqüentemente, a ativação líquida para exhibições futuras, a operação de soma  $w(t+1) = w(t) + \eta x(t)$  ajusta o vetor de pesos rotacionando-o para mais próximo do vetor de entrada  $x(t)$ , reduzindo o ângulo  $\alpha$  entre eles de modo que  $\cos(\alpha)$  (e a projeção) aumente.
- Quando ocorre  $e(t) = -1$ , significa que o neurônio gerou uma ativação forte indevida ( $y(t) = +1$  mas  $d(t) = 0$  ou  $-1$ ). Geometricamente, o vetor de pesos está indevidamente apontado na direção da entrada. A operação de subtração  $w(t+1) = w(t) - \eta x(t)$  força o vetor de pesos a se afastar espacialmente de  $x(t)$  (aumentando o ângulo  $\alpha$  entre eles de forma que o cosseno e a projeção diminuam).
- Essas duas condições direcionais separadas se fundem em uma única equação compacta:  $w(t+1) = w(t) + \eta e(t)x(t)$ . O sinal do próprio erro  $e(t) \in \{-1, +1\}$  assume o papel de ditar automaticamente se haverá soma (correção de aproximação) ou subtração (correção de afastamento) espacial, automatizando a convergência da fronteira de decisão sem a necessidade de condicionais separadas.

## 17. Conexão entre Divisão de Dados, Critérios de Parada e Generalização

- a) O objetivo de utilizar critérios de parada baseados em um limiar inferior de erro de treinamento ( $J(t) < J_{min}$ ) é encerrar o ajuste quando a rede adquiriu o conhecimento estatístico geral do mapeamento de entrada-saída. Contudo, se o algoritmo continuar rodando indefinidamente sem monitoração (por exemplo, ultrapassando um número excessivo de épocas), ele pode se especializar demais no conjunto específico de treino. O monitoramento por épocas  $K_{max}$  atua como uma barreira reguladora para prevenir overtraining.

- b) O erro de generalização obtido no teste é tipicamente maior que o erro de treinamento porque o conjunto de teste agrupa padrões "não vistos" durante a fase de modificação de parâmetros. No treinamento, os pesos são explicitamente orientados para minimizar as divergências locais do conjunto ajustado; no teste, avalia-se apenas a capacidade de interpolação do conhecimento para cenários reais inéditos. A cada época de treinamento, a boa prática exige misturar e alterar de forma aleatória a ordem de apresentação dos padrões (slide 35). Isso é crucial para que o algoritmo do gradiente descendente não fique ciclando de forma determinística por rotas espaciais repetitivas de mínimos locais induzidos puramente pela sequência dos dados. A aleatorização desfaz qualquer viés sequencial nas atualizações recursivas locais.

## Parte V — Questões de Explicação Aprofundada

### 18. Derivação Teórica Passo a Passo do Algoritmo Backpropagation (Camada de Saída)

- a) A equação recursiva de minimização iterativa do erro pelo método do gradiente descendente para o  $k$ -ésimo neurônio de saída é dada por (slide 60):

$$m_k(t+1) = m_k(t) - \eta \frac{\partial J(t)}{\partial m_k(t)}$$

O termo correspondente à derivada parcial  $\frac{\partial J(t)}{\partial m_k(t)}$  representa o gradiente da função de custo instantânea. Ele indica a direção de maior crescimento do erro no espaço dos parâmetros de peso. Subtrair esse gradiente ponderado pela taxa de aprendizagem  $\eta$  significa orientar a variação dos pesos no sentido oposto de maior declive para alcançar de forma iterativa um ponto de mínimo.

- b) Usando a regra da cadeia, decomponemos a derivada total em relação ao vetor de pesos  $m_k$  fatorando o impacto de variações sucessivas em cada bloco funcional (slide 61):

$$\frac{\partial J(t)}{\partial m_k(t)} = \frac{\partial J(t)}{\partial e_k(t)} \cdot \frac{\partial e_k(t)}{\partial y_k(t)} \cdot \frac{\partial y_k(t)}{\partial a_k(t)} \cdot \frac{\partial a_k(t)}{\partial m_k(t)}$$

- c) Cálculo explícito de cada uma das quatro parcelas (conforme detalhado no slide 62):

(a) Primeiro termo:  $\frac{\partial J(t)}{\partial e_k(t)} = \frac{\partial}{\partial e_k(t)} \left( \frac{1}{2} \sum e_j^2(t) \right) = e_k(t)$ .

(b) Segundo termo: como  $e_k = d_k - y_k$ , temos  $\frac{\partial e_k(t)}{\partial y_k(t)} = \frac{\partial (d_k - y_k)}{\partial y_k(t)} = -1$ .

(c) Terceiro termo: como  $y_k = \varphi(a_k)$ , temos  $\frac{\partial y_k(t)}{\partial a_k(t)} = \varphi'(a_k(t)) = y'_k(t)$ .

(d) Quarto termo: como  $a_k(t) = m_k^T(t)z(t)$ , o vetor gradiente em relação ao vetor de pesos resulta na própria entrada da camada:  $\frac{\partial a_k(t)}{\partial m_k(t)} = z(t)$ .

Multiplicando os quatro termos encontrados:

$$\frac{\partial J(t)}{\partial m_k(t)} = e_k(t) \cdot (-1) \cdot y'_k(t) \cdot z(t) = -e_k(t)y'_k(t)z(t)$$

Definindo o gradiente local da camada de saída por  $\delta_k(t) = e_k(t)y'_k(t)$ , substituímos o produto de volta na equação recursiva de gradiente descendente para obter:

$$m_k(t+1) = m_k(t) - \eta(-\delta_k(t)z(t)) = m_k(t) + \eta\delta_k(t)z(t)$$

### 19. O Mecanismo de Erro "Inventado" do Backpropagation para a Camada Oculta

- a) Como a camada oculta de neurônios intermediários está embutida na topologia da rede, não há valores de saídas desejadas externas  $d_i$  correspondentes para calcular um termo de erro de forma convencional. A solução encontrada dita que a própria matriz de conexões de pesos de saída  $M$  seja utilizada para propagar as divergências da rede em sentido reverso. Desta forma, o erro "inventado" para o neurônio oculto  $i$ , denotado por  $e_i(t)$  no slide 57, é a soma ponderada de todos os gradientes locais de erro  $\delta_k(t)$  da camada subsequente de saída, multiplicados pelos correspondentes pesos de conexões  $m_{ki}(t)$  que os interconectam:

$$e_i(t) = \sum_{k=1}^c m_{ki}(t) \delta_k(t)$$

Esse termo quantifica a parcela de responsabilidade do neurônio oculto  $i$  sobre cada um dos erros cometidos na camada de saída.

- b) O gradiente local  $\delta_i(t)$  do neurônio oculto é formado pelo produto do erro retropropagado  $e_i(t)$  com a correspondente derivada da sua função de ativação local sigmoide  $z'_i(t)$  (slide 57):
- Para a sigmoide logística:  $\delta_i(t) = e_i(t) \cdot z'_i(t) = e_i(t) \cdot z_i(t) (1 - z_i(t))$ .
  - Para a tangente hiperbólica:  $\delta_i(t) = e_i(t) \cdot z'_i(t) = e_i(t) \cdot 0,5 (1 - z_i^2(t))$ .
- c) Escreva o passo a passo da regra final de atualização dos pesos ocultos  $w_i(t+1) = w_i(t) + \eta \delta_i(t) x(t)$  demonstrando de que forma as modificações dependem estritamente do vetor de entrada expandido original  $x(t)$ .

## 20. Análise Matemática de Dimensionamento com a Heurística de Baum-Haussler

- a) Para uma rede  $MLP(p, q_1, c)$ , o número total de parâmetros livres e ajustáveis  $Z$  é expresso pela fórmula (slide 72):

$$Z = (p+1)q_1 + (q_1+1)c$$

Substituindo os valores conhecidos de variáveis de entrada  $p = 10$  e neurônios de saída  $c = 2$ , temos:

$$Z = (10+1)q_1 + (q_1+1)2 = 11q_1 + 2q_1 + 2 = 13q_1 + 2$$

- b) Substituindo a expressão de  $Z$  e os valores de  $N = 1000$  e do erro tolerado no teste  $\epsilon = 0,1$  na inequação de Baum-Haussler:

$$N > \frac{Z}{\epsilon} \Rightarrow 1000 > \frac{13q_1 + 2}{0,1}$$

Multiplicando ambos os lados por 0,1:

$$1000 \cdot 0,1 > 13q_1 + 2 \Rightarrow 100 > 13q_1 + 2$$

Subtraindo 2 de ambos os lados:

$$98 > 13q_1 \Rightarrow q_1 < \frac{98}{13} \approx 7,538$$

Sendo o número de neurônios ocultos  $q_1$  necessariamente um valor inteiro, o número máximo absoluto de neurônios ocultos permitidos para satisfazer a restrição estatística de Baum-Haussler é de:

$$q_1^{max} = 7 \text{ neurônios ocultos.}$$

- c) Se utilizarmos um número de neurônios ocultos superior a este limite (por exemplo,  $q_1 = 15$ ), o número correspondente de parâmetros livres excederá consideravelmente a capacidade de generalização suportada pelo volume finito de dados ( $N = 1000$ ). Uma rede superdimensionada se comportará de forma excessivamente complexa, aproximando-se com facilidade a um cenário de superajuste (overfitting). A rede "memorizará" as flutuações e ruídos específicos contidos nos dados de treinamento, apresentando alto erro quando testada com novas observações coletadas no ambiente real.

## Tabela Resumo dos Exercícios

| No. | Conteúdo Avaliado                            | Tipo de Questão | Habilidade Cognitiva         | Dificuldade |
|-----|----------------------------------------------|-----------------|------------------------------|-------------|
| 1   | Mapeamento biológico no neurônio MP          | Conceitual      | Compreensão / Tradução       | Fácil       |
| 2   | Ativações suaves e série de Taylor           | Conceitual      | Análise teórica              | Média       |
| 3   | Saturação e paralisia da rede                | Conceitual      | Rigor dedutivo               | Difícil     |
| 4   | Teoremas de Hornik e linearização            | Conceitual      | Síntese matemática           | Difícil     |
| 5   | Análise algébrica/geométrica da porta OR     | Análise/Interp. | Cálculo algébrico / Esboço   | Fácil       |
| 6   | Inseparabilidade do XOR / espaço oculto      | Análise/Interp. | Análise de espaço            | Média       |
| 7   | Passo-a-passo do Perceptron Simples          | Análise/Interp. | Cálculo iterativo de pesos   | Média       |
| 8   | Funções-objetivo Perceptron vs. Adaline      | Análise/Interp. | Comparação conceitual        | Difícil     |
| 9   | Completar código R do Perceptron             | Computacional   | Implementação e depuração    | Fácil       |
| 10  | Implementar regra LMS matricial em R         | Computacional   | Álgebra de matrizes em R     | Média       |
| 11  | Normalização linear unipolar/bipolar         | Computacional   | Implementação de algoritmos  | Média       |
| 12  | Código de derivadas da sigmóide logística    | Computacional   | Codificação de funções       | Média       |
| 13  | Correção de bug no backpropagation R         | Computacional   | Correção de erros sintáticos | Difícil     |
| 14  | Cosseno, ângulo e ativação de pesos          | Conexão         | Associação de conceitos      | Fácil       |
| 15  | Número total de parâmetros e viés-variância  | Connection      | Cálculo estrutural e teoria  | Média       |
| 16  | Casos geométricos de ajuste vs. regra do PS  | Connection      | Interpretação algorítmica    | Média       |
| 17  | Parada do treino e capacidade de generalizar | Connection      | Conexão metodológica         | Difícil     |
| 18  | Dedução formal do backpropagation de saída   | Explicação      | Dedução matemática rigorosa  | Difícil     |
| 19  | Erro retropropagado para camadas ocultas     | Explicação      | Formulação passo-a-passo     | Difícil     |
| 20  | Inequação de Baum-Haussler para generalizar  | Explicação      | Cálculo limitador e análise  | Difícil     |